On the Compatibility of Non-Uniform Quantization Schemes with OPTQ-Based LLM Compression

Abstract

The increasing scale of large language models (LLMs) has significantly increased memory and computational requirements during inference. These constraints can be relaxed with model quantization, which reduces the precision of a model's weights, thereby lowering the memory footprint and latency. We focus on OPTQ, a state-of-the-art weight-only post-training model quantization method, which was originally demonstrated with asymmetric uniform min-max quantization and claims compatibility with any quantization approach. While uniform quantization is regarded as more hardware efficient, non-uniform methods, albeit slower, better represent the distribution of the data to quantize. This raises the question of whether using non-uniform quantization in OPTQ could increase accuracy at the cost of increased latency.

In this report, we address this question by theoretically and empirically analyzing the compatibility non-uniform quantization methods with OPTQ, while also exploring potential speedups with custom kernel implementations. Our compatibility analysis suggests non-uniform methods make OPTQ's error correction more unstable and our empirical results on five prominent non-uniform quantization methods suggest that the performance degradation is generally acceptable.

We conclude that the OPTQ framework is best tailored for uniform quantization methods and does not adequately support the majority of non-uniform methods, contradicting the original author's claim. Future work could investigate suitable error correction techniques for non-uniform approaches, creating a non-uniform quantization method tailored to OPTQ, and further exploring kernel optimizations for latency improvements.

1 Introduction

As LLMs become more complex over time, their parameter count has increased exponentially, officially reaching the orders of billions[1]. However, with an increased number of parameters comes steep computational limitations. For instance, if Deepseek's recent R1[1] model's 671 billion parameters were 32-bit floating points[1], the model would require approximately 2.684TB of memory just to store the parameters. Even with state-of-the-art GPUs (e.g, A100/H100 with 80GB of RAM), simply running inference on this model would require a large, costly multi-GPU deployment. To address this computational constraint, recent research has focused on compressing the model with minimal accuracy loss. This report will focus on one of these approaches, *model quantization*, which compresses the model by reducing each parameter's bit-size.

Model Quantization. *Quantization* refers to reducing the number of bits used to represent a high-precision value (e.g., FP32) by using a low-precision alternative (e.g., INT8). In the context of model compression, this refers to quantizing model parameters, notably weights and activations. For a full overview of other LLM compression approaches, see [12].

Model quantization is generally categorized into two main types: in *quantization-aware training (QAT)*, the quantized trained model is finetuned to mitigate the accuracy degradation arising from the quantization procedure; in *post-training quantization (PTQ)*, the trained model's parameters

[1]This is purely an example; in reality, Deepseek R1 is an exception to the rule - it was trained using smaller, mixed precision parameters. Nevertheless, the overall industry shows a preference 32-bit floating point parameters.

are quantized in one shot, excluding finetuning. Although less accurate than QAT, PTQ is computationally faster, as there is no retraining involved[12]. We will be focusing on PTQ.

Post-training quantization can be categorized into three groups: *weight-activation quantization*, *weight-only quantization*, and *key-value cache quantization*. Weight-activation methods quantize both weights and activations, while weight-only quantization methods focus exclusively on weights. Although weight-only approaches can achieve lower bit-width than the more sensitive weight-activation methods, they must be dequantized when multiplied with the full-precision activations, resulting in a computational overhead during inference[10]. Key-value cache quantization of an LLM's attention layers, reduces the memory consumption when processing input streams with long tokens.

OPTQ. OPTQ[2], one of the strongest weight-only post-training quantization methods to date, seeks a quantized weight matrix $\hat{\mathbf{W}}$ that solves the following mean squared error objective relative to the original weight matrix $\mathbf{W}$:

$$\text{argmin}_{\hat{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\|_2^2 \tag{1}$$

We detail OPTQ in Algorithm 1. First, the Cholesky decomposition of the weight matrix $\mathbf{W}$'s Hessian $H^{-1} = (2\mathbf{X}\mathbf{X}^\intercal + \lambda\mathbf{I})$ is stored. Then, $\mathbf{W}$ is split into B blocks of columns vectors. In each block, the leftmost full-precision column vector $\mathbf{W}_{:,j}$ is quantized. The subsequent weights $\mathbf{W}_{:,j(i+B)}$ within the block are updated to factor in the resulting quantization error $\mathbf{E}_{:,j-i}$. Once all columns within the block have been quantized, the remaining weights $\mathbf{W}_{:,(i+B)}$ are also updated to factor in the quantization errors. This process is repeated until all weights have been quantized.

Algorithm 1 OPTQ given inverse Hessian $\mathbf{H}^{-1} = (2\mathbf{X}\mathbf{X}^\intercal + \lambda\mathbf{I})^{-1}$ and blocksize B

$\mathbf{Q} \leftarrow \mathbf{0}_{d_{row} \times d_{col}}$
$\mathbf{E} \leftarrow \mathbf{0}_{d_{row} \times B}$
$\mathbf{H}^{-1} \leftarrow \text{Cholesky}(\mathbf{H}^{-1})^\intercal$
for i = 0, B, 2B, ... **do**
  for j = i, ..., i + B - 1 **do**
    $\mathbf{Q}_{:,j} \leftarrow Q(\mathbf{W}_{:,j})$ ▷ Quantize the weight $\mathbf{W}_{:,j}$
    $\mathbf{E}_{:,j-i} \leftarrow (\mathbf{W}_{:,j} - \mathbf{Q}_{:,j})/[\mathbf{H}^{-1}]_{jj}$
    $\mathbf{W}_{:,j:(i+B)} \leftarrow \mathbf{W}_{:,j:(i+B)} - \mathbf{E}_{:,j-i} \cdot \mathbf{H}^{-1}_{j,j:(i+B)}$
  end for
  $\mathbf{W}_{:,(i+B):} \leftarrow \mathbf{W}_{:,(i+B):} - \mathbf{E} \cdot \mathbf{H}^{-1}_{i:(i+B),(i+B):}$
end for

Accuracy-wise, this method can reduce a model's bitwidth to 3 or 4 bits per weight with negligible loss in accuracy. Speed-wise, OPTQ can provide computational speedups of 3.25 with high-end GPUs (NVIDIA A100) and 4.5 with cost-effective GPUs (NVIDIA A6000). In fact, it can quantize 175-billion parameter GPT models in approximately four GPU hours. This is especially helpful for large models, as those are typically harder to quantize with minimal accuracy loss.

Scalar Parameter Quantization. OPTQ uses *scalar quantization*, specifically asymmetric uniform min-max quantization. Scalar quantization reduces the size of the parameters with a scale factor which relies on a step size s. It can be split into two categories: **uniform quantization** and **non-uniform quantization**. In uniform quantization, each full-precision parameter x_F is mapped

linearly such that their quantization levels[2] $Q(x_F)$ are uniformly spaced. We say its step size is constant. Non-uniform quantization maps the parameters x_F in a non-linear fashion, making the step size between the quantization levels $Q(x_F)$ non-uniformly spaced. Since the step size depends on the distribution of the full-precision parameters, it is more accurate than uniform quantization.

A quantization method's range can be either **symmetric** or **asymmetric**. In symmetric quantization, the range is defined as $[-a, a]$ where $\text{quant}(0) = 0$. In asymmetric quantization, the range is bounded between two values a and b such that $|a| \neq |b|$. In this case, we have a corresponding zero-factor z for which $\text{quant}(0) = z$.

Motivation. OPTQ was evaluated using asymmetric uniform min-max quantization and claims to perform well on other quantization methods. Since non-uniform methods can better represent the distribution of values to quantize, we seek to investigate whether this notion could improve the accuracy of OPTQ with a reasonable tradeoff on efficiency.

2 Technical Details

In this section, we begin by explaining the quantization method used by OPTQ, and then analyze the theoretical and empirical applicability of non-uniform methods to OPTQ. Note that in OPTQ, quantization methods are applied row-wise, i.e., on the weight vectors $\mathbf{w}$ of $\mathbf{W}$.

Asymmetric Uniform Min-Max Quantization. OPTQ uses asymmetric uniform min-max quantization. It works as follows: given a weight vector $\mathbf{w}$, the maximum possible quantization level[3] $q_{\max}$, set $w_{\min} = \min \mathbf{w}$ and $w_{\max} = \max \mathbf{w}$. If $\min \mathbf{w} = \max \mathbf{w} = 0$, then $w_{\min} = -1$ and $w_{\max} = 1$.

Then, for the uniform quantization and dequantization operations

$$\text{quant}_U(w) = \text{clip}(\text{round}(\tfrac{w}{s}) + z_s, 0, q_{\max}) = q \qquad \text{dequant}_U(q) = s \times (q - z_s) \tag{2}$$

where $q \in [0, q_{\max}]$, we compute the scale s and zero-factor z such that:

$$s = (w_{\max} - w_{\min})/q_{\max} \qquad z_s = -w_{\min}/s \tag{3}$$

2.1 Theoretical Applicability of Non-Uniform Methods

Recall that uniform methods use linear mappings, whereas non-uniform methods use non-linear mappings. Then, to estimate the impact of different quantization schemes, we can analyze the quantization error.

For a given $w \in \mathbb{R}$ and quantization translation operation $Q : \mathbb{R} \to \mathbb{R}$ where

$$Q(w) = \text{dequant}(\text{quant}(w)) \tag{4}$$

we analyze the signed error ε_q incurred from quantizing the weight w_q:

$$\varepsilon_q = w_q - Q(w_q) \tag{5}$$

[2]Representable value to which a set of full-precision values are mapped to.
[3]i.e. value from the quantization grid representing a set of full-precision values.

Deriving the OPTQ Update Operation. To understand how the error ε_q is corrected by the update step (Algorithm 1 l.6-8) and the conditions on the error, we expand upon the derivation from Hassibi et al[3] with the row-wise variant of our update optimization problem (Equation 1):

$$\text{argmin}_{\hat{\mathbf{w}}}\mathcal{L}(\hat{\mathbf{w}}) = \text{argmin}_{\hat{\mathbf{w}}}\|\mathbf{w}X - \hat{\mathbf{w}}X\|_2^2 \tag{6}$$

Taking the Taylor expansion of $\mathcal{L}(\hat{\mathbf{w}})$ where δ is the penalty vector:

$$\mathcal{L}(\hat{\mathbf{w}}) = \mathcal{L}(\mathbf{w}+\delta) = \underbrace{\mathcal{L}(\mathbf{w})}_{0} + \underbrace{\frac{\partial\mathcal{L}(\mathbf{w})}{\partial\mathbf{w}}}_{0}\cdot\delta + \tfrac{1}{2}\delta^\intercal\mathbf{H}\delta = \tfrac{1}{2}\delta^\intercal\mathbf{H}\delta \tag{7}$$

We also have the constraint $Q(w_q) = w_q + \varepsilon_q$ which can be more broadly defined as:

$$\mathbf{e}_q^T\cdot\delta + w_q = Q(w_q) \tag{8}$$

where $\mathbf{e}_q$ is the unit vector corresponding to w_q.

We solve the optimization problem $\text{argmin}_{\hat{\mathbf{w}}}\mathcal{L}(\hat{\mathbf{w}})$ given the constraint Equation 8 by forming the corresponding Lagrangian:

$$\mathcal{L}(\delta,\lambda) = \tfrac{1}{2}\delta^\intercal\mathbf{H}\delta - \lambda(\mathbf{e}_q^T\cdot\delta + w_q - Q(w_q)) \tag{9}$$

with the Lagrangian multiplier λ. Getting the derivative with respect to δ:

$$\tfrac{\partial\mathcal{L}(\delta,\lambda)}{\partial\delta} = \mathbf{H}\delta - \lambda\cdot\mathbf{e}_q \tag{10}$$

Setting $\frac{\partial\mathcal{L}(\delta,\lambda)}{\partial\delta} = 0$ and solving for δ:

$$\delta = \lambda\mathbf{e}_q\mathbf{H}^{-1} \tag{11}$$

Moreover, recalling Equation 8, we solve for δ:

$$\delta = \tfrac{Q(w_q)-w_q}{\mathbf{e}_q^\intercal} \tag{12}$$

Plugging Equation 11 and Equation 12 together and solving for λ, we get:

$$\lambda = \tfrac{-Q(w_q)-w_q}{\mathbf{e}_q^\intercal\mathbf{H}^{-1}\mathbf{e}_q} = \tfrac{-Q(w_q)-w_q}{[\mathbf{H}^{-1}]_{qq}} \tag{13}$$

where $\mathbf{e}_q^\intercal\mathbf{H}^{-1}\mathbf{e}_q$ returns the qth diagonal of $\mathbf{H}^{-1}$.

We finally inject Equation 13 into Equation 11 to solve δ:

$$\delta = -\tfrac{(Q(w_q)-w_q)}{\mathbf{H}^{-1}_{qq}}\cdot\mathbf{e}_q\cdot\mathbf{H}^{-1} = \boxed{-\tfrac{(Q(w_q)-w_q)}{[\mathbf{H}^{-1}]_{qq}}\cdot\left(\mathbf{H}^{-1}\right)_{:,q}} \tag{14}$$

Recall in Algorithm 1 (l.8-10) that the penalty δ is applied to all unquantized weights in $\mathbf{w}$. To prevent the penalized weights from over/undershooting, the error correction procedure and thus the quantization error must be stable such that the error upholds the following criterion:

- the error ε is **bounded**.
- the mean $\mathbb{E}[\varepsilon]$ is **zero**.
- the variance $\mathbb{V}[\varepsilon]$ is **low across subsets of data to quantize**(here, weights).

With these conditions, we can analyze the stability of uniform and non-uniform quantization.

Statistical Stability of Uniform Quantization. Recalling Equation 2, if $Q(\cdot)$ is a uniform operation, then

$$Q(w_q) = \text{dequant}_U(\text{quant}_U(w_q)) = s \cdot \text{round}\left(\frac{w_q}{s}\right) \tag{15}$$

and since the step size is uniform, the probability density function $\rho(\varepsilon)$ is $\rho(\varepsilon) = \frac{1}{s}$ First, let us estimate the bounds on ε. We can decompose it as:

$$\varepsilon_q = w_q - s \cdot \text{round}\left(\frac{w_q}{s}\right) = s \cdot \left(\frac{w_q}{s} - \text{round}\left(\frac{w_q}{s}\right)\right) \tag{16}$$

Since the round$(\cdot)$ function performs standard nearest integer rounding such that $\forall a \in \mathbb{R}$ $a - \text{round}(a) \in \left[-\frac{1}{2}, \frac{1}{2}\right)$, then:

$$\varepsilon_q \in \left[-\frac{s}{2}, \frac{s}{2}\right) \tag{17}$$

As this bound is independent from the quantized weight w_q, we can generalize it:

$$\boxed{\varepsilon \in \left[-\frac{s}{2}, \frac{s}{2}\right)} \tag{18}$$

Since ε is symmetrically bounded, the mean error is trivially:

$$\boxed{\mathbb{E}\left[\varepsilon\right] = 0} \tag{19}$$

Finally, deriving the variance $\mathbb{V}\left[\varepsilon\right]$ of the error:

$$\mathbb{V}\left[\varepsilon\right] = \mathbb{E}\left[\left(\mathbb{E}\left[\varepsilon\right] - \varepsilon\right)^2\right] = \mathbb{E}\left[\varepsilon^2\right] = \int_{-s/2}^{s/2} \varepsilon^2 d\rho(\varepsilon) = \frac{1}{s} \cdot \left[\frac{\varepsilon^3}{3}\right]_{-s/2}^{s/2} = \boxed{\frac{s^2}{12}} \tag{20}$$

We have shown that for any weight vector $\mathbf{w}$, the quantization error is bounded, has zero mean, and its variance is also bounded, decreasing quadratically with smaller scales s. Therefore, we can deduce that the error induced by uniform quantization is stable, making uniform quantization compatible with OPTQ.

Statistical Stability of Non-Uniform Quantization. If we attempt the same for non-uniform methods, we arrive at an impasse; non-uniform methods, by definition, have nonlinear step sizes which are determined by the weight distribution of $\mathbf{w}$. Since the step size affects $Q(\cdot)$ and consequently ε, then the error bound, the mean $\mathbb{E}\left[\varepsilon\right]$ and the variance $\mathbb{V}\left[\varepsilon\right]$ depend more heavily on the data distribution than in uniform quantization. As such, they can only be estimated empirically on a per-method basis - an approach which, if performed over all individual weight vectors of an LLM's layers, is computationally intractable. Moreover, certain data distributions may yield a large quantization error, causing the error correction procedure to overshoot. Therefore, the error introduced by non-uniform quantization is not stable, making non-uniform methods a theoretically less reliable form of quantization when used in combination with OPTQ.

2.2 Practical Applicability of Non-Uniform Methods

We have shown that non-uniform methods theoretically lead to a greater instability in error correction. In this section, we describe the non-uniform quantization techniques used to empirically assess the magnitude of this instability in practice. Recall that all of these methods, similarly to asymmetric uniform min-max quantization, are applied on the weight vectors $\mathbf{w}$ of a layer.

Quantile-based Quantization. Quantile-based quantization is an asymmetric non-uniform quantization method where full-precision values are placed into bins depending on the weight quantile they are closest to. The quantiles are determined based on the quantization precision (i.e., the number of bits available in quantized format). So, for instance, if the quantization precision is 3, then the weights would be distributed into 8 quantiles.

Logarithmic Quantization. Logarithmic quantization is a non-uniform quantization technique where floating-point numbers are mapped to the nearest power of a chosen base (typically 2). Equation 21 shows the formula for base-2 quantization.

$$\text{quant}(x) = \text{Sign}(x) \cdot 2^{\text{round}(log_2|x|)} \tag{21}$$

Unlike uniform minmax quantization (as used in [2]) which uses evenly spaced levels, The possible quantization levels are powers of the base (e.g. 0.25, 0.5, 1, 2, 4, ... for base 2) and spaced logarithmically. These levels are denser near zero and become progressively sparser further away from zero.

This non-uniformity can potentially more accurately represent the typical weight distribution of LLMs, which often have many values clustered near zero. Naturally, logarithmic quantization provides higher precision for these smaller values.

To constrain the quantization level to a specific bit width b, the range of possible exponents ($\text{round}_2|x|$) is limited. Typically, this is done by defining a minimum and maximum permitted exponent on the data range and the desired number of levels. We define 2^{b-1} desired positive levels and then clamp the calculated exponent within this range before applying the final power operation ($2^{\text{round}(log_2|x|)}$).

Logarithmic quantizations adds the opportunity to replace expensive multiplication operations with efficient bitshift operations [4]. For example, $w_i \cdot a_i$ translates to $a_i \ll \log_2 |w_i|$.

Additive Powers of Two (APoT) Quantization. Additive Powers-of-Two (APoT) quantization [5] is another non-uniform quantization technique designed to better represent the distribution of weights. Instead of mapping values to a single power-of-two like logarithmic quantization, APoT represents numbers as the sum of a small, fixed number (k) of signed powers-of-two terms. Our implementation specifically uses $k = 2$.

It begins by establishing a dynamic range for quantization through a clipping threshold α, which is determined empirically from the weight distribution. Any input value x is first clipped to the range $[-\alpha, \alpha]$:

$$x_{clipped} = \text{clip}(x, -\alpha, \alpha) = \max(-\alpha, \min(x, \alpha)) \tag{22}$$

Based on this threshold α and target bit width b, a range of permissible integer exponents $[p_{min}, p_{max}]$ is defined. p_{max} is typically related to $\lfloor \log_2 \alpha \rfloor$, while p_{min} depends on p_{max} and the number of quantization levels from bit width b.

These exponents define the base set of signed powers-of-two:

$$P = s \cdot 2^p \mid s \in -1, 1, p \in \mathbb{Z}, p_{min} \leq p \leq p_{max} \tag{23}$$

The defining feature of APoT quantization is the formation of the quantization levels L. For $k = 2$, these levels are formed by summing pairs of terms from the base set P (Equation 23), ensuring the result sums remain within the clipping boundary $[-\alpha, \alpha]$:

$$L = l \mid l = p_1 + p_2, p_1 \in P \cup 0, p_2 \in P \cup 0, |l| \leq \alpha \tag{24}$$

This construction yields a non-uniform set of levels L that are denser near zero. The final quantization function quant(x) maps the clipped input to the nearest level L_j in the generated set L:

$$\text{quant}(x) = \underset{L_j \in L}{\text{argmin}} |x_{clipped} - L_j| \tag{25}$$

By employing sums ($k = 2$), APoT can generate a richer set of quantization levels compared to single power-of-two methods for a given exponent range, potentially enhancing precision. The reliance on powers-of-two gives the possibility of efficient hardware implementation through bit-shifts and additions [5]. The bit width b primarily influences the exponent range $[p_{min}, p_{max}]$, thereby controlling the dynamic range and granularity captured by the quantization levels.

K-Means Quantization. K-Means quantization is another non-uniform quantization scheme that adapts quantization levels to specific distribution of the weights. Unlike methods with predefined levels (e.g. uniform or logarithmic), K-Means employs the K-Means clustering algorithm to determine an optimal set of quantization levels, known as centroids, directly from the data it aims to quantize.

The core idea is to partition the n input data points into $X = \{x_1, x_2, ..., x_n\}$ into k clusters, where k is determined by the desired bit width b (typically $k = 2^b$). Each cluster is represented by its centroid, and these k centroids form the quantization levels. The K-Means algorithm iteratively refines the centroid positions to minimise the in-cluster MSE.

The iterative process involves two steps. First, the assignment step, where each data point x_i is assigned to the cluster $S_j^{(t)}$ whose current centroid $c_j^{(t)}$ is nearest (in terms of Euclidean distance).

$$S_j^{(t)} = x_i \mid |x_i - c_j^{(t)}|^2 \leq |x_i - c_l^{(t)}|^2 \quad \forall l, 1 \leq l \leq k \tag{26}$$

Then, the update step, where centroid are recalculated as the mean of all data points assigned to their respective clusters.

$$c_j^{(t+1)} = \frac{1}{|S_j^{(t)}|} \sum_{x_i \in S_j^{(t)}} x_i \tag{27}$$

These steps are repeated until the centroids converge or a maximum number of iterations is reached. The final set of centroids $C = \{c_1, c_2, ..., c_k\}$ constitutes the set of quantization levels.

Once the centroids C are determined, the quantization function quant(x) maps any given input value x to the nearest centroid in the set C:

$$\text{quant}(x) = \underset{c_j \in C}{\text{argmin}} |x - c_j|^2 \tag{28}$$

As the centroids are derived directly from the distribution, K-Means quantization can potentially achieve a lower quantization error compared to fixed quantization schemes. There is a computational overhead associated with the iterative centroid finding process.

Lloyd-Max Quantization. In Lloyd-Max's quantization, introduced by [6][7], weights are categorized into bins based on the decision boundaries b_{i-1} and b_i, and then assigned the bin's corresponding quantization level a_i. Similarly to K-Means, these decision boundaries and quantization levels are iteratively refined to better represent the distribution of weight values. We detail the algorithm below.

Given M arbitrary quantization levels a_i where $i \in [0, M-1]$ and a user-defined maximum number of epochs:

1. For each $i \in [1, M-1]$, set the decision boundaries b_i such that $b_i = \frac{1}{2}(a_{i+1} + a_i)$.
2. For each $i \in [1, M-1]$, update the quantization levels a_i such that $a_i = \mathbb{E}[x_i]$ where $b_{i-1} \leq x_i < b_i$ and b_0 and b_M are considered as $-\infty$ and $+\infty$ respectively.
3. Repeat steps 1 and 2 until convergence or the maximum number of epochs is reached.

By repeatedly refining the decision boundaries and the corresponding quantization levels, Lloyd-Max reduces the MSE (mean squared error) between the weights and their corresponding quantization levels, thereby better representing the distribution of weight values.

2.3 Custom kernel implementation

While our primary aim was to evaluate different quantization implementations within the GPTQ framework, realising the memory footprint benefits of 3/4-bit quantization necessitates *bit-packing*. Storing each 4-bit quantized weight individually in standard data types like FP16 or INT8 would negate the potential memory savings.

Generic Kernel. As such, we designed and implemented a custom CUDA kernel that would pack eight 4-bit quantized weights into a 32-bit integer. These weights would be unpacked on-the-fly by the kernel and dequantized to a floating point (FP16) format. Then, they would multiply the dequantized weights to the corresponding input vector element before being accumulated as a result. This approach was generic; it did not optimise for specific quantization schemes.

Logarithmic Variation. With the knowledge of logarithmic quantization being capable of using efficient bit-shift operations, we wrote a specific logarithmic kernel implementation (LogPack4bit) to take advantage of this. This kernel handles the packed weight format and performs the matrix-vector multiplication using computationally efficient bit-shift operations, avoiding floating-point arithmetic for the core computation.

Weights are stored packed, with each 8-bit byte holding two 4-bit LogPack4bit codes. For each 4-bit code processed, the kernel performs the following core steps:

1. **Unpack & Decode:** Extracts the 4-bit code from the packed byte and decodes its sign (from the MSB) and 3-bit exponent index (from the LSBs).
2. **Compute & Bit-Shift:** Unmaps the 3-bit index to the actual integer exponent using a layer-specific minimum. The multiplication by the corresponding 8-bit quantized activation is then executed via bit shifts and applying the decoded sign.
3. **Accumulate:** Adds the result of the bit-shift operation to a running accumulator.

After processing all weights for an output feature, a block-level reduction sums the partial results. The final accumulated value is scaled by the activation quantization factor, and the bias is added before writing the FP32 output.

3 Evaluation

Setup. We based our implementation on the original OPTQ [2] codebase [4], which evaluated its methods on the OPT [11] and BLOOM [9] model families. Our setup consisted of a cluster of four NVIDIA Quadro RTX 6000 GPUs (24GB VRAM each), with experiments only having access to a single GPU. LloydMax was implemented with 1 iteration, as it performed worse on increasing iterations. K-means was implemented with at most 100 iterations.

3.1 Quantization Schemes

To assess the impact of different PTQ techniques on model performance, we evaluated several 4-bit quantization schemes across a range of OPT and BLOOM model sizes. We evaluated on on standard language modelling benchmarks (WikiText-2, PTB, C4) with **perplexity**, where lower scores indicate better language modelling capability and thus less information loss due to quantization.

OPT Results. The results for OPT model quantization are presented in Table 1. Some quantization schemes failed to produce a result due to out of memory, as indicated by $--$ in the tables, due to the methods additional data structures exceeding the total GPU memory.

Dataset	Quantizer	125M	350M	1.3B	2.7B	6.7B	13B
WikiText-2	Uniform MinMax	**30.28**	24.00	**15.09**	**12.88**	**11.04**	**10.21**
	APOT	64.32	**22.31**	83.12	17.76	156.57	359.99
	LloydMax	37.37	24.48	15.49	13.13	11.18	–
	Logarithm	36.05	24.29	16.63	13.56	11.33	10.48
	Quantile	33.85	23.79	15.91	13.47	11.52	10.37
	K-means	44.22	23.06	20.33	13.71	13.07	–
PTB	Uniform MinMax	**36.81**	28.69	**17.77**	**15.79**	**13.35**	**12.50**
	APOT	72.81	**26.69**	108.85	24.14	337.58	253.91
	LloydMax	44.14	29.86	18.90	16.31	13.64	–
	Logarithm	46.95	29.86	20.00	16.53	13.82	12.82
	Quantile	42.21	30.15	19.32	17.11	14.09	12.82
	K-means	53.06	27.83	35.61	16.78	16.57	–
C4	Uniform MinMax	**27.00**	22.75	**15.49**	**13.70**	**12.04**	**11.35**
	APOT	51.53	**21.12**	96.56	20.92	189.19	449.82
	LloydMax	32.40	23.55	16.13	14.24	12.22	–
	Logarithm	32.80	23.22	16.96	14.41	12.33	11.55
	Quantile	30.36	23.25	16.61	14.70	12.62	11.59
	K-means	38.12	21.99	25.30	15.01	14.66	–

Table 1: Comparison of perplexity scores for OPT models across different datasets. All quantization is 4-bit with groupsize 1024. Lower scores indicate better performance.

[4] https://github.com/ist-daslab/gptq

Across all three datasets, the original Uniform Min-Max quantization scheme consistently demonstrated strong performance, particularly for OPT models with 1.3B parameters and larger.

The Lloyd-Max quantizer also performed competitively, often yielding perplexity scores very close to Uniform Min-Max, especially on the WikiText-2 and C4 datasets. It represents a viable alternative, though results were unavailable for the largest (13B) model, as a result of the GPU memory being exceeded with the higher memory overhead required for the quantization scheme.

Logarithm and Quantile quantization methods provided stable and reasonable performance across all model sizes and datasets. While generally resulting in slightly higher perplexity than Uniform Min-Max or Lloyd-Max, they consistently outperformed the more erratic schemes and showed predictable scaling behaviour as model size increased.

In contrast, APoT exhibited highly variable performance. While achieving the best result for the OPT-350M model, it performed significantly worse, often dramatically so, on other model sizes.

K-means quantization also showed mixed results. It performed well for the OPT-350M model but yielded considerably higher perplexity for the 1.3B model before improving again at larger scales. Similar to Lloyd-Max, results were unavailable for the 13B model.

The relative performance ranking of the quantizers remained largely consistent across the WikiText-2, PTB, and C4 datasets for the OPT model family.

BLOOM Results. Evaluation was also conducted on the BLOOM model family to show that particular quantization schemes were not biased to OPT.

Quantizer	560M	1.1B	1.7B	3B	7.1B
Uniform MinMax	22.99	18.05	**15.68**	**13.63**	**11.44**
APOT	28.18	22.79	18.13	17.09	14.90
LloydMax	**22.89**	**17.98**	15.69	13.69	**11.44**
Logarithm	25.32	19.74	17.03	14.44	12.09
Quantile	23.18	18.23	15.75	13.76	11.49
K-means	26.58	22.20	18.20	14.65	13.06

Table 2: Perplexity scores for BLOOM models on WikiText-2 dataset across different model sizes and quantization schemes. All quantization is 4-bit with groupsize 1024. Lower scores indicate better performance.

Evaluation on the BLOOM model family revealed slightly different performance dynamics on the WikiText-2 dataset. Lloyd-Max emerged as the most effective quantizer for BLOOM models, achieving the lowest perplexity for most model sizes tested (560M, 1.1B, 1.7B, 7.1B).

Uniform Min-Max and Quantile quantization were also highly competitive for BLOOM, with scores closely trailing LloydMax. Notably, APoT quantization, while not the best performing, showed much greater stability on BLOOM compared to OPT, without the extreme perplexity increases observed on the larger OPT models. This improved stability might be related to the inherent architectural differences in BLOOM. Logarithm and K-means generally lagged behind the other methods for the BLOOM architecture.

3.2 4-bit Kernel

We aimed to evaluate the effectiveness of the 4-bit quantization schemes when implemented in a custom CUDA kernel. First, we implemented a generic 4-bit kernel before investigating an optimised

variant for logarithmic quantization.

Generic 4-bit Kernel. We evaluated the generic 4-bit kernel with bit-packing in terms of latency against the baseline OPT model, as seen in Table 3.

Model	Median Latency (ms)	Speedup on baseline (×)
Baseline Model (FP16)	6.322	1.0
4-bit Kernel (Quantile)	9.381	0.67

Table 3: Comparison of opt-125m latencies on the base model and the Quantile-quantized model with our 4-bit kernel implementation.

The observed degraded latency is likely due to the optimised nature of FP16 matrix multiplication on GPUs compared to our 4-bit operations. This is further discussed in section 4.1.

Logarithmic Variation. Here, we evaluate the proposed LogPack4bit quantization scheme, which utilises a custom CUDA kernel for matrix-vector multiplication, applied to the OPT-125m model.

Prior to full model integration, we benchmarked the custom kernel with bit-shifts was evaluated in isolation against standard *torch.matmul* using matrix dimensions representative of the opt-125m model. We observed that our kernel's bit-shift approach would have a latency of 0.192 ms compared to 0.828 ms for FP16 matrix multiplication, showing a speedup of 4.31×.

However, this benchmark did not yield to real world performance. The median latency per token generation was measured for the baseline opt-125m FP16 model and the custom kernel for logarithmic quantization. As shown in Table 4, the results from the full model benchmark contrast sharply with the promising operation benchmark, achieving 55% of the average latency of the base model. We discuss this result further in section 4.1.

Model	Median Latency (ms)	Speedup on baseline (×)
Baseline Model (FP16)	6.322	1.0
LogPack4bit	11.555	0.55

Table 4: Comparison of opt-125m latencies on the base model and the logarithmic quantized model with our custom kernel implementation.

Model Size Reduction. We observed a model size deccrease for the logarithmic kernel implementation of nearly 53% (251 MB → 118 MB), thanks to the significant decrease in storage space required when packing two 4-bit weights into a byte.

The generic 4-bit kernel received a model size reduction of 132 MB. While less than the logarithmic variant, this 47% decrease is still significant. The general approach to packing weights means that not all layers are possible to be packed in 4 bits, and thus must remain as FP16 in some cases.

4 Conclusion and Discussion

We discuss the implications of our results, analyze the shortcomings of our analysis, and conclude our report by considering future works.

4.1 Discussion

We have shown that the majority of non-uniform quantization schemes are not compatible with OPTQ. Specifically, we discuss the theoretical applicability, the empirical performance, and the potential speedups.

Overfitting of Weights. We originally expected non-uniform methods to outperform asymmetric uniform min-max quantization due to their improved ability to describe the distribution of weights accurately. However, in our experiments, we consistently found that non-uniform methods often underperformed compared to their uniform counterpart. After investigating the statistical properties of OPTQ's weight update step, we found that it was the heavy reliance on the data distribution which was increasing perplexity - since the step sizes are adapted to the weight distribution, they become highly specific. This leads to an increase in the variability of the error correction step and lessened robustness across different data inputs. This behaviour can be interpreted as a form of overfitting to the model weights, where the quantization becomes so tailored to the specific distribution of the weights that it loses information captured during training, such as generalization capacity.

Empirical Performance of Non-Uniform Methods. Empirically, this results in an overall increase in perplexity, whose instability across model sizes heavily depends on the scheme employed. Despite the findings on the theoretical potential for instability in error correction when combining non-uniform schemes with the OPTQ framework, we found that certain methods do not differ greatly from the baseline Uniform MinMax implementation. While on the whole the uniform approach outperformed the non-uniform methods in perplexity terms, Lloyd-Max, Logarithmic and Quantile quantization performed closely to the uniform perplexity scores, particularly at larger model sizes.

We were surprised by the variable performance of APoT quantization. While it surpassed the baseline on all datasets in the OPT-350m model variant, it otherwise consistently scored significantly worse than any other quantization implementation in every other case. Meaning, we consider this particular performance as an anomaly.

The results for the BLOOM model showed similar findings, with Lloyd-Max quantization actually slightly out performing the uniform approach in some model sizes, and otherwise performing very close. We suspect this is caused by the particular model architecture of BLOOM, but are unable to verify our claims due to time constraints.

Lloyd-Max also shows close scores to the uniform baseline for OPT, but they are particularly degraded in lower model sizes. This difficulty to quantize smaller models was noted in the original OPTQ paper [2].

Overall, Lloyd-Max shows the most potential out of all the non-uniform methods.

Kernel Implementation. In both the generic and logarithmic 4-bit kernel implementations, the median latency was worse than the baseline. This largely due to the FP16 baseline leveraging highly optimised libraries (cuBLAS) that make extensive use of NVIDIA Tensor Cores (specialised hardware units designed for accelerating matrix multiplication). In the logarithmic kernel, the bit-shift operations must take place on general purpose CUDA cores as opposed to the specialised Tensor Cores.

Also, the bit-packed kernels require dequantizing every weight before multiplication which is computationally expensive. To achieve better results, specific kernels for the quantization technique would

need to be written to avoid this step, and instead try and use dot products on weights directly on the quantized weights.

Furthermore, factors like memory access patterns and instruction efficiency within the custom kernel itself may contribute to its lower performance compared to the heavily optimised FP16 libraries.

Overall, the 4-bit kernels show potential to significantly reduce the memory overhead required to run these models. However, further work is needed on making the inference latency optimal. The Quadro RTX 6000 GPUs used in the experiment are not the state-of-the-art, and lack the capability of having INT4 precision on Tensor Cores, unlike the NVIDIA A100 GPUs used in OPTQ [2]. We suspect that without this limitation, our kernels would perform significantly better.

4.2 Limitations

To better assess the impact of non-uniform quantization with OPTQ, our analysis could have further explored the following areas.

Insufficient Method-Level Differentiation in Compatibility Analysis. In our experiments, we observed that the performance of the non-uniform methods varies a lot based on methods. For instance, while the perplexity of APoT quantization increases exponentially with model size, Lloyd-Max typically closely follows the perplexity of asymmetric uniform min-max quantization. Clearly, there exist trends in perplexity which are more specific to each quantization method beyond they being non-uniform. A per-method statistical analysis could have given more insight into the algorithm-specific properties that influence the compatibility of a non-uniform method with OPTQ. Additionally, this could also have brought insight into how the number of iterations might have influenced the performance of iterative non-uniform quantization methods (e.g., LloydMax, K-Means).

Quantification of Accuracy-Efficiency Tradeoff. We began this project with the understanding that non-uniform methods would be slower and use more memory. This is because they use less efficient hardware operations and require larger per-weight-vector data structures. An analysis of the estimated latency and memory usage could have quantified the per-method accuracy-efficiency tradeoff, offering a more concrete comparison between the uniform min-max quantization scheme and the non-uniform quantization schemes.

Model and Hardware Limitations. We were limited in experiments by the 24 GB VRAM present on Quadro RTX 6000 GPUs, meaning we could not experiment on OPT variants beyond 13B or the full size 176B BLOOM model. This likely limited the success of our quantization approaches, as we observed the trend of perplexity differences being much lower in larger models. Also, since BLOOM and OPT, many more LLMs have been released, of which we have not evaluated the success of our quantization methods on.

4.3 Future Work

Given our results and observed shortcomings, we detail directions for future work. Specifically, we focus on finding new error correction techniques, new quantization methods, exploring new constraints (smaller bitwidth or group size), and more in-depth kernel optimizations.

Method-Level Error Correction Techniques. In our compatibility analysis, we observed that in general, non-uniform methods are incompatible with OPTQ, specifically in the error correction step. Possible research directions would be to derive alternative error correction strategies that might be more compatible with specific non-uniform quantization methods. For example, in the logarithmic non-uniform approach, the step size depends on the magnitude of the weight being quantized. As a result, quantization levels for larger magnitudes cover a wider range of values, leading to a loss in accuracy. This could be addressed by scaling the intensity of the error correction based on the magnitude of the quantized value.

Creating a Non-Uniform Quantization Method Tailored to OPTQ. In our limitations, we highlighted the possibility of identifying and analyzing the properties of individual non-uniform quantization methods that significantly influence their performance and accuracy. By leveraging the beneficial properties and avoiding those that increase perplexity, it may be possible to tailor a non-uniform method compatible with OPTQ which outperforms uniform min-max quantization.

Group Size. All empirical experiments were conducted with the same group size (1024), which is the number of weights quantized at each step. It would prove interesting to vary the group size and examine how non-uniform methods perform.

3-bit Quantization. Here, we only experimented with 4-bit non-uniform quantization. Future work could study the effectiveness of 3-bit non-uniform quantization within OPTQ.

Latency Improvements with Kernel Optimizations. As discussed, further optimizations can be made to reduce the overhead of dequantizing in our custom kernel implementation. A kernel can be purpose-built for specific quantization methods, with operations more suited to the schemes parameters (as seen in the logarithmic variant). Spatial locality in the 4-bit kernels needs to be considered further also. Furthermore, writing a kernel for newer architectures with INT4 precision on Tensor Cores would likely see significant latency improvements.

5 Generative AI Statement

In this project, we used Generative AI alongside the literature [8] to identify a sufficiently broad range of non-uniform quantization schemes commonly applied in weight-only model quantization. It was also used in an assistive role with the code, particularly with the kernel implementation.

6 Reproducibility Statement

In the supplementary materials, we provide code to reproduce the experiments in this report. Note that our code is originally an extension of the OPTQ[2] codebase. It includes:

- Implementations of Quantile, Logarithmic, APoT, K-Means and Lloyd-Max for weight-only row-wise quantization.
- Extension of OPT and BLOOM model family compression to include different quantization methods.
- Custom CUDA 11.7 kernels for 4-bit quantization, with a special logarithmic variant.

References

[1] DeepSeek-AI et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948 [cs]. Jan. 2025. DOI: 10.48550/arXiv.2501.12948. URL: http://arxiv.org/abs/2501.12948 (visited on 04/14/2025).

[2] Elias Frantar et al. "OPTQ: ACCURATE POST-TRAINING QUANTIZATION FOR GENERATIVE PRE-TRAINED TRANSFORMERS". en. In: (2023).

[3] B. Hassibi, D.G. Stork, and G.J. Wolff. "Optimal Brain Surgeon and general network pruning". In: *IEEE International Conference on Neural Networks*. Mar. 1993, 293–299 vol.1. DOI: 10.1109/ICNN.1993.298572. URL: https://ieeexplore.ieee.org/document/298572 (visited on 05/04/2025).

[4] Edward H. Lee et al. "LogNet: Energy-efficient Neural Networks Using Logarithmic Computation". In: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Mar. 2017, pp. 5900–5904. DOI: 10.1109/ICASSP.2017.7953288. (Visited on 04/29/2025).

[5] Yuhang Li, Xin Dong, and Wei Wang. *Additive Powers-of-Two Quantization: An Efficient Non-uniform Discretization for Neural Networks*. Feb. 2020. DOI: 10.48550/arXiv.1909.13144. arXiv: 1909.13144 [cs]. (Visited on 05/03/2025).

[6] S. Lloyd. "Least squares quantization in PCM". In: *IEEE Transactions on Information Theory* 28.2 (Mar. 1982), pp. 129–137. ISSN: 1557-9654. DOI: 10.1109/TIT.1982.1056489. URL: https://ieeexplore.ieee.org/document/1056489 (visited on 05/06/2025).

[7] J. Max. "Quantizing for minimum distortion". In: *IRE Transactions on Information Theory* 6.1 (Mar. 1960), pp. 7–12. ISSN: 2168-2712. DOI: 10.1109/TIT.1960.1057548. URL: https://ieeexplore.ieee.org/document/1057548 (visited on 05/06/2025).

[8] Babak Rokh, Ali Azarpeyvand, and Alireza Khanteymoori. "A Comprehensive Survey on Model Quantization for Deep Neural Networks in Image Classification". In: *ACM Trans. Intell. Syst. Technol.* 14.6 (Nov. 2023), 97:1–97:50. ISSN: 2157-6904. DOI: 10.1145/3623402. URL: https://dl.acm.org/doi/10.1145/3623402 (visited on 04/01/2025).

[9] BigScience Workshop et al. *BLOOM: A 176B-Parameter Open-Access Multilingual Language Model*. June 2023. DOI: 10.48550/arXiv.2211.05100. arXiv: 2211.05100 [cs]. (Visited on 04/30/2025).

[10] Zhewei Yao et al. *ZeroQuant-V2: Exploring Post-training Quantization in LLMs from Comprehensive Study to Low Rank Compensation*. arXiv:2303.08302 [cs] version: 3. May 2023. DOI: 10.48550/arXiv.2303.08302. URL: http://arxiv.org/abs/2303.08302 (visited on 04/24/2025).

[11] Susan Zhang et al. *OPT: Open Pre-trained Transformer Language Models*. June 2022. DOI: 10.48550/arXiv.2205.01068. arXiv: 2205.01068 [cs]. (Visited on 04/30/2025).

[12] Xunyu Zhu et al. "A Survey on Model Compression for Large Language Models". In: *Transactions of the Association for Computational Linguistics* 12 (Nov. 2024), pp. 1556–1577. ISSN: 2307-387X. DOI: 10.1162/tacl_a_00704. URL: https://doi.org/10.1162/tacl_a_00704 (visited on 04/01/2025).